OmniSync

Mutual Exclusion Visualizer

Code Explained — A Beginner's Guide

This document explains every file of the OmniSync project in plain English. No prior experience with JavaScript or Node.js is required.

Files covered:

api.js · app.js · server.js

Table of Contents

#	Section	File
1	What is Mutual Exclusion?	—
2	Project Overview — How the Files Connect	—
3	api.js — The Messenger	api.js
4	app.js — The Brain of the UI	app.js
5	server.js — The Coordinator	server.js
6	How They All Work Together	—
7	Quick Reference — Endpoints	server.js

1 · What is Mutual Exclusion?

Imagine a single bathroom in an office. Many people (processes) want to use it, but only **one person can be inside at a time**. The lock on the door ensures that. In computers, this idea is called **Mutual Exclusion**.

When two or more programs (called *processes*) share the same resource (a file, a printer, a database row), they must take turns. If two programs write to the same file at the same moment, the file gets corrupted. Mutual Exclusion prevents that.

The Centralized Coordinator Model

OmniSync uses the simplest and most intuitive approach:

- **One central authority** (the Coordinator / server) controls who gets access.
- **Clients** ask the coordinator for permission.
- The coordinator says **GRANT** (go ahead) or **WAIT** (queue up).
- After finishing, the lock is released and the next client in line gets it.

> ■ *Think of it like a teacher who controls the class microphone. Students raise their hand (REQUEST), the teacher picks one (GRANT), and when done they hand the mic back (RELEASE).*

2 · Project Overview — How the Files Connect

The project has three JavaScript files. Each has a very clear job:

File	Where it runs	Job
api.js	Browser	Sends HTTP requests to the server
app.js	Browser	Controls what the user sees and responds to button clicks
server.js	Node.js (your computer)	Runs the mutex algorithm and responds to requests

The Journey of a Single Click

Here is exactly what happens when a user clicks **Request Access** on Client-1:

- **app.js** detects the button click.
- **app.js** calls a function in **api.js** to send a network request.
- **api.js** sends a POST request to **http://localhost:5000/api/client/Client-1/request**.
- **server.js** receives that request, runs the mutex algorithm, and replies with the result.
- **api.js** gives the result back to **app.js**.
- **app.js** updates the page — changes badges, the lock label, and the console log.

3 · api.js — The Messenger

Think of **api.js** as the *phone* that app.js uses to call the server. It knows the server's address and how to make each type of call. It doesn't do any thinking — it just sends messages and returns the replies.

The Base URL

The file starts by defining the server's address:

■ api.js

```
const API_CONFIG = {
  baseUrl: 'http://localhost:5000/api',
};
```

localhost means *this computer*. Port **5000** is where the server listens. **/api** is the prefix for all data routes. This one line is the only place you need to change if you deploy to a real server.

Function 1 — requestClient(clientId)

Called when a client wants to enter the critical section.

■ api.js

```
async requestClient(clientId) {
  const res = await fetch(
    `http://localhost:5000/api/client/${clientId}/request`,
    { method: 'POST' }
  );
  return await res.json();   // gives back the server's answer
}
```

async / await — JavaScript's way of waiting for a slow operation (a network call) without freezing the page.

fetch() — the browser's built-in function to send HTTP requests.

POST — the type of HTTP request used to *send data or trigger an action* (like submitting a form).

res.json() — converts the server's text reply into a JavaScript object so the code can read individual fields like *response.data.currentHolder*.

Function 2 — crashCoordinator()

Tells the server to simulate a failure. No data is sent — it is just a signal.

■ **api.js**

```
async crashCoordinator() {
  const res = await fetch(
    'http://localhost:5000/api/coordinator/crash',
    { method: 'POST' }
  );
  return await res.json();
}
```

Function 3 — resetSystem()

Asks the server to wipe all state and start fresh.

■ **api.js**

```
async resetSystem() {
  const res = await fetch(
    'http://localhost:5000/api/reset',
    { method: 'POST' }
  );
  return await res.json();
}
```

■ *All three functions follow the same pattern: build the URL → fetch → return JSON. This consistency is intentional — it makes the file easy to extend if you add more endpoints later.*

4 · app.js — The Brain of the UI

app.js is the largest file. It controls everything the user sees and reacts to every button click. Think of it as the *remote control* for the web page. It has four main responsibilities:

- Show pop-up modals (information dialogs).
- Add messages to the live log terminal.
- Re-draw the page whenever the server state changes.
- Listen for button clicks and call the right api.js function.

4.1 · The Two Global Variables

■ app.js

```
let startTime    = Date.now();  // when the page loaded (ms since 1970)
let isSimRunning = false;       // is the simulation currently running?
```

startTime is used to show elapsed time in the log (e.g. *[342 ms] Client-1 sent REQUEST*).

isSimRunning tracks whether the user clicked Start — it toggles the button label.

4.2 · showModal() — The Pop-up Controller

This function controls the single shared modal dialog used throughout the app. It is flexible enough to show success messages, errors, alerts, and information.

■ app.js

```
function showModal({ type, title, messageHtml, confirmText, onConfirm }) {
  // 1. Find the modal elements in the HTML
  const modal      = document.getElementById('customModal');
  const confirmBtn = document.getElementById('modalConfirmBtn');

  // 2. Fill in the title and body text
  document.getElementById('modalTitle').textContent = title;
  document.getElementById('modalBody').innerHTML    = messageHtml;

  // 3. Change the button colour depending on the type
  confirmBtn.className = type === 'error' ? 'btn btn-red' : 'btn btn-green';

  // 4. Show the modal
  modal.classList.add('flex');

  // 5. Close it when the user clicks OK or the X
  confirmBtn.addEventListener('click', () => modal.classList.remove('flex'), { once
: true });
}
```

document.getElementById() — finds an HTML element by its id attribute.

innerHTML — sets the HTML content inside an element.

classList.add/remove — adds or removes a CSS class (which shows/hides the modal).

{ once: true } — the event listener fires only once and then removes itself automatically.

4.3 · logMessage() — Writing to the Terminal

■ app.js

```
function logMessage(text, packetType, sender, receiver) {
  const terminal = document.getElementById('logTerminal');
  const elapsed  = Date.now() - startTime;  // how many ms since start

  const entry = document.createElement('div');   // create a new line
  entry.className  = 'log-entry';
  entry.textContent = `[${elapsed} ms] ${text}`;

  // Clicking the line opens the packet inspector modal
  entry.onclick = () => showPacketModal(packetType, sender, receiver, text);

  terminal.appendChild(entry);       // add to the terminal
  terminal.scrollTop = terminal.scrollHeight;  // scroll to bottom
}
```

document.createElement('div') — creates a brand new HTML element in memory.

appendChild() — attaches that new element as a child of the terminal div.

scrollTop = scrollHeight — auto-scrolls the terminal to always show the latest line.

4.4 · renderState() — Updating the UI

Every time the server responds, **renderState()** is called with the latest state object. It updates every visual widget — the lock tag, the client badges, and the metric numbers.

■ app.js

```
function renderState(state) {
  const holder = state.currentHolder || 'None';

  // Update the numbers at the bottom of the CS card
  document.getElementById('activeHolderVal').textContent = holder;
  document.getElementById('queueLengthVal').textContent  = state.waitingQueue.lengt
h + ' Clients';

  // Flip the lock tag
  const lockTag = document.getElementById('lockIconContainer');
  if (holder !== 'None') {
    lockTag.textContent = 'LOCKED';
```

```
    lockTag.className   = 'lock-tag locked';    // green blinking border
  } else {
    lockTag.textContent = 'UNLOCKED';
    lockTag.className   = 'lock-tag unlocked';  // grey border
  }

  // Update each client badge (idle / waiting / in CS)
  ['Client-1', 'Client-2', 'Client-3'].forEach((id, i) => {
    const badge  = document.getElementById(`client${i+1}Badge`);
    const status = state.clients[id];
    if (status === 'IN_CRITICAL_SECTION') badge.className = 'status-badge active';
    else if (status === 'WAITING')        badge.className = 'status-badge waiting'
;
    else                                  badge.className = 'status-badge idle';
  });
}
```

4.5 · DOMContentLoaded — Setting Up Button Listeners

All button listeners are wrapped inside **DOMContentLoaded** so the code only runs *after* the full HTML page has loaded.

■ app.js

```
document.addEventListener('DOMContentLoaded', () => {

  // Start / Pause button
  document.getElementById('toggleSimBtn').addEventListener('click', () => {
    isSimRunning = !isSimRunning;    // flip the flag
    // ... update button text and badge colour
  });

  // All three 'Request Access' buttons share one listener
  document.querySelectorAll('.clientReqBtn').forEach(button => {
    button.addEventListener('click', async (e) => {
      const clientId = e.target.getAttribute('data-client');  // 'Client-1' etc.
      const response = await MutexAPI.requestClient(clientId);
      renderState(response.data);   // redraw the UI
    });
  });

  // Crash and Reset buttons call their api.js counterparts
  // ... (same pattern)
});
```

querySelectorAll('.clientReqBtn') — finds all elements with that CSS class (all three Request buttons). One listener handles all three by reading the **data-client** attribute to know which client was

clicked.

async (e) => { ... await ... } — the click handler is async because it needs to wait for the server to reply before updating the UI.

5 · server.js — The Coordinator

This is the **Node.js backend**. It runs on your computer (not in the browser) and is the only place where the mutex algorithm is actually executed. Think of it as the *judge* who decides which client gets the shared resource.

5.1 · Setup — Express and CORS

■ server.js

```
const express = require('express');   // web server framework
const cors    = require('cors');      // allows browser to reach this server
const path    = require('path');      // builds file paths correctly on any OS

const app  = express();
const PORT = 5000;

app.use(express.json());  // read JSON bodies from incoming requests
app.use(cors());          // allow requests from the browser

// Serve the HTML/CSS/JS files so the browser can open the app
app.use(express.static(path.join(__dirname, '..', 'ME(GUI)')));
```

require() — Node.js's way of importing a library (like import in Python).

express — a lightweight framework that makes creating HTTP routes easy.

cors — without this, the browser blocks requests to a different origin (a security rule called Same-Origin Policy). CORS middleware lifts that block.

express.static() — tells the server to *serve files from a folder*. Opening http://localhost:5000 will now show index.html automatically.

5.2 · The State Object — Single Source of Truth

All coordinator data lives in one JavaScript object in memory:

■ server.js

```
let state = {
  currentHolder: null,         // which client holds the lock right now
  waitingQueue:  [],           // FIFO list of clients waiting
  clients: {
    'Client-1': 'IDLE',        // possible values: IDLE | WAITING | IN_CRITICAL_SEC
TION
    'Client-2': 'IDLE',
    'Client-3': 'IDLE',
  },
```

```
  crashed:      false,         // true after /coordinator/crash is called
  releaseTimer: null,          // handle for the auto-release setTimeout
};
```

This object is the *single source of truth* — every route reads from it and writes to it. The server never reads from a database; it only uses this in-memory object. That means restarting the server resets everything.

5.3 · grantToNext() — Serving the Queue

■ server.js

```
function grantToNext() {
  if (state.waitingQueue.length === 0) return;  // nobody waiting, do nothing

  const nextClient = state.waitingQueue.shift();  // take the FIRST in line (FIFO)
  state.currentHolder       = nextClient;
  state.clients[nextClient] = 'IN_CRITICAL_SECTION';

  // Auto-release after 3 seconds
  state.releaseTimer = setTimeout(() => {
    state.clients[nextClient] = 'IDLE';
    state.currentHolder       = null;
    state.releaseTimer        = null;
    grantToNext();   // check the queue again (recursive-ish chaining)
  }, 3000);
}
```

array.shift() — removes and returns the *first* element of the array. This is what makes the queue FIFO (First-In, First-Out). The client that asked first always gets served first.

setTimeout(fn, 3000) — schedules fn to run after 3000 milliseconds (3 seconds). This simulates the client doing work inside the critical section and then leaving.

5.4 · Route 1 — POST /api/client/:clientId/request

This is the most important route. It runs the full mutex decision logic.

■ server.js

```
app.post('/api/client/:clientId/request', (req, res) => {
  const { clientId } = req.params;   // e.g. 'Client-1'

  // Guard 1: reject unknown clients
  if (!state.clients.hasOwnProperty(clientId))
    return res.status(400).json({ success: false, message: 'Unknown client' });

  // Guard 2: reject if coordinator has crashed
```

```
  if (state.crashed)
    return res.status(503).json({ success: false, message: 'Coordinator is DOWN' })
;

  // Guard 3: ignore duplicate requests
  if (state.clients[clientId] !== 'IDLE')
    return res.status(409).json({ success: false });

  if (state.currentHolder === null) {
    // CS is FREE → grant immediately
    state.currentHolder       = clientId;
    state.clients[clientId]   = 'IN_CRITICAL_SECTION';
    // schedule auto-release in 3 seconds ...
  } else {
    // CS is BUSY → put in queue
    state.waitingQueue.push(clientId);
    state.clients[clientId]  = 'WAITING';
  }

  res.json({ success: true, data: state });   // send state back to browser
});
```

:clientId — a URL parameter. If the URL is */api/client/Client-2/request*, then req.params.clientId equals 'Client-2'.

res.status(400) — sets the HTTP status code. 400 = Bad Request, 503 = Service Unavailable, 409 = Conflict. The browser's fetch() checks this.

res.json() — sends a JavaScript object back as a JSON text response.

5.5 · Route 2 — POST /api/coordinator/crash

■ server.js

```
app.post('/api/coordinator/crash', (req, res) => {
  state.crashed = true;   // flip the flag — that's it!
  res.json({ success: true, message: 'Coordinator has crashed.' });
});
```

This demonstrates the **single point of failure** weakness of the centralized model. One flag flip stops the entire system. In a real distributed system, this is why designers add redundancy (backups, leader election, etc.).

5.6 · Route 3 — POST /api/reset

■ server.js

```
app.post('/api/reset', (req, res) => {
```

```
  if (state.releaseTimer) clearTimeout(state.releaseTimer);  // cancel any pending timer
  state = createInitialState();   // replace state with a fresh blank copy
  res.json({ success: true, data: state });
});
```

clearTimeout() is critical here. If we didn't cancel the pending timer and just overwrote state, the old timer would fire 3 seconds later and corrupt the brand new state object. Always cancel timers before resetting.

5.7 · Starting the Server

■ server.js

```
app.listen(PORT, () => {
  console.log(`OmniSync server running at http://localhost:${PORT}`);
});
```

app.listen() starts the HTTP server and tells it to accept connections on port 5000. The callback function (the arrow function) runs once when the server is ready — that is when the console message appears.

6 · How They All Work Together

Here is the full sequence diagram of one complete mutex cycle — from button click to UI update:

Step	Who acts	What happens
1	User	Clicks 'Request Access' on Client-2
2	app.js	Detects the click via addEventListener
3	app.js	Calls MutexAPI.requestClient('Client-2')
4	api.js	Sends POST http://localhost:5000/api/client/Client-2/request
5	server.js	Receives request; checks if CS is free or busy
6	server.js	CS is free → sets currentHolder='Client-2', starts 3s timer
7	server.js	Replies with { success:true, data: { currentHolder:'Client-2', ... } }
8	api.js	Returns the parsed JSON to app.js
9	app.js	Calls renderState(response.data)
10	app.js	DOM updates: lock tag = LOCKED, Client-2 badge = IN CRITICAL SECTION
11	server.js	3 seconds later: auto-releases lock, calls grantToNext()
12	server.js	If queue empty → waits. If queue has Client-3 → grants it

■ *The browser (app.js + api.js) only sees step 7 — the server's reply. Steps 11 and 12 happen entirely on the server side. The UI does not automatically update when the auto-release fires; it updates on the next button click that triggers another renderState() call.*

7 · Quick Reference — Endpoints

All three API routes are POST requests (no data is needed in the body — the action is encoded in the URL itself):

Endpoint	HTTP	What it does	Success response
`/api/client/{id}/request`	`POST`	`Request CS access`	`{ success, data }`
`/api/coordinator/crash`	`POST`	`Crash the coordinator`	`{ success, message }`
`/api/reset`	`POST`	`Reset all state to IDLE`	`{ success, data }`

HTTP Status Codes Used

Code	Name	When it appears in this project
200	OK	Every successful response
400	Bad Request	Unknown clientId (not Client-1/2/3)
409	Conflict	Client already in CS or already waiting
503	Service Unavailable	Request sent after coordinator crashed

How to Run the Full Project

■ Terminal

```
# Step 1 – go into the server folder
cd MutualExclusiondA/server

# Step 2 – install dependencies (only needed once)
npm install

# Step 3 – start the server
npm start

# Step 4 – open the app in your browser
# Navigate to:  http://localhost:5000
```

■ *You do NOT need a separate Live Server extension anymore. The Node.js server serves both the HTML files AND the API — everything runs on port 5000.*